

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 3

Контейнеризация написанного приложения средствами
Docker

Выполнил:

Соболев Артём

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Реализовать Dockerfile для каждого сервиса
2. Написать общий docker-compose.yml
3. Настроить сетевое взаимодействие между сервисами

Ход работы

Для всех сервисов реализован единый параметризованный Dockerfile. Конкретный сервис выбирается через build-аргумент SERVICE в docker-compose.yml. Такой подход позволяет не дублировать одинаковую логику сборки для каждого микросервиса, но при этом каждый сервис собирается в отдельный Docker-образ.

Dockerfile состоит из двух этапов сборки. На первом этапе используется образ eclipse-temurin:21-jdk, так как для сборки Java-приложения требуется JDK. В контейнер копируются файлы проекта, после чего выполняется команда сборки нужного Gradle-модуля. На втором этапе используется образ eclipse-temurin:21-jre. В него копируется только готовый .jar-файл из нужного сервиса. После этого приложение запускается командой. Такой подход уменьшает размер итогового образа, так как в runtime-контейнер не попадают Gradle, исходный код и временные файлы сборки.

Для запуска всего приложения был написан общий docker-compose.yml. В нём описаны контейнеры всех микросервисов. Сетевое взаимодействие между контейнерами настроено средствами Docker Compose. Все сервисы находятся в одной внутренней сети и могут обращаться друг к другу по имени сервиса. Для асинхронного взаимодействия между микросервисами используется RabbitMQ. Он также запускается как отдельный контейнер и доступен другим сервисам по имени.

Для корректного порядка запуска используются `depends_on` и `healthcheck`. Базы данных и RabbitMQ сначала проверяются на готовность, и только после этого запускаются сервисы, которые от них зависят. Например, `user-service` запускается после того, как контейнер `user-db` перейдёт в состояние `healthy`.

Вывод

В результате была выполнена контейнеризация микросервисного приложения. Для сервисов реализован единый параметризованный `Dockerfile`, который позволяет собирать отдельный Docker-образ для каждого микросервиса без дублирования одинаковой логики сборки. Также был написан общий `docker-compose.yml`, в котором описаны микросервисы, отдельные базы данных PostgreSQL и RabbitMQ. Сетевое взаимодействие между сервисами настроено через внутреннюю сеть Docker Compose, где контейнеры обращаются друг к другу по именам сервисов.